

软件体系结构详细设计初稿

小组成员：龚陈陈、王婷、覃雨欣、周飞凤

序号	姓名	学号	职责	分工(本学期)
1	龚陈陈	2024091203018	组长	文档撰写/后端
2	王婷	2024091201030	组员	web端
3	覃雨欣	2024091201029	组员	安卓端
4	周飞凤	2024091201028	组员	后端

1. 引言

1.1 编写目的

本报告旨在详细描述"多数据风控系统（贷款业务）"的软件体系结构设计，明确系统的类结构、关键类之间的交互关系、数据模型及服务实现方式。本设计为开发团队提供实现依据，确保系统开发、测试和维护的一致性。

1.2 读者对象

本报告面向系统开发人员、测试人员及项目管理人员。

1.3 软件系统概述

本系统是一个面向互联网金融场景的个人贷款风控系统，支持用户在线申请小额贷款。系统通过整合多源数据（如征信、运营商、社保、行为日志等），利用规则引擎与机器学习模型进行风险评估，实现自动化审批与额度定价。目标用户包括金融机构风控部门与终端借款人。

2. 软件设计约束和原则

2.1 软件设计约束

- 运行环境**：系统部署在Linux服务器，支持高并发访问
- 技术栈**：Spring Boot 3.0+, Java 17, MySQL 8.0, Redis
- 接口规范**：RESTful API, JSON数据格式
- 性能要求**：贷款申请审批响应时间 ≤ 2 秒，支持每秒1000+并发请求
- 安全要求**：符合金融行业安全规范，支持OAuth2.0认证

2.2 软件设计原则

- 分层架构**：表现层、服务层、数据层三层分离
- 高内聚低耦合**：每个服务模块职责单一，接口清晰
- 可扩展性**：支持未来接入第三方风控模型
- 可维护性**：模块化设计，代码结构清晰
- 容错性**：关键业务流程有完善的异常处理机制

3. 详细设计方案

3.1 系统体系结构设计

系统采用分层架构设计，核心服务层包含多个独立服务模块，通过HTTP/JSON协议通信。整体架构如下：



关键设计说明：

- 系统采用"决策中心"模式，由ApplicationService作为协调者，聚合多个风控子服务结果
- AuthService作为核心认证服务，提供资产认证、银行账户认证、征信认证等能力
- RiskAssessService作为风险评估核心，整合AIApproveService和ManualApproveService

3.2 类设计

3.2.1 LoanProduct (贷款项目)

属性：

```
private Long id;

@NotBlank(message = "产品名称不能为空")
@Size(max = 100, message = "产品名称不能超过100个字符")
private String productName;

@Size(max = 500, message = "产品描述不能超过500个字符")
private String description;

@Size(max = 200, message = "贷款用途说明不能超过200个字符")
private String loanUsage;

@NotNull(message = "最短期数不能为空")
@Min(value = 1, message = "最短期数不能小于1")
private Integer minTerm;

@NotNull(message = "最长期数不能为空")
@Min(value = 1, message = "最长期数不能小于1")
private Integer maxTerm;

@NotNull(message = "期数步长不能为空")
@Min(value = 1, message = "期数步长不能小于1")
```

```
private Integer termStep;

@Size(max = 1000, message = "促销详情不能超过1000个字符")
private String promotionDetails;

private LocalDateTime createTime;
private LocalDateTime updateTime;
```

方法:

```
// 所有的setter和getter方法
public void setter(Object object);
public Object getter();
```

设计说明: LoanProduct类是贷款产品核心类, 封装了产品配置信息和业务逻辑, 通过LoanProductService提供服务接口。

3.2.2 Order (用户已贷款项目)

属性:

```
private Long id;
private Long userId;
private Long productId;

private OrderStatus status; // "NORMAL", "OVERDUE", "SETTLED"
private BigDecimal repaidAmount;
private BigDecimal loanAmount;
private BigDecimal interestRate;

private RepaidType repaidType; // "等额本息"、"等额本金"、"先息后本"、"一次性还清"
private Integer loanPeriod;
private Integer term;
private Integer currentTerm;
private String contract; // 合同路径

private Integer overdueDays;

private LocalDateTime startTime;
```

方法:

```
// 所有的setter和getter方法
public void setter(Object object);
public Object getter();
```

设计说明: Loan类是用户贷款的核心实体, 记录了贷款的详细状态和还款信息, 通过OrderService提供服务。

3.2.3 AuthService (认证服务层)

属性:

```
private UserService userService; // 用户认证信息类
private ThirdService thirdService; // 三方认证信息类,可能多个,这里统称为ThirdService
private ValuationService valuationService; // 计算贷款分数的机器学习模型API类
```

方法:

```
// 资产认证
void assetAuth();

// 银行账户认证
void bankAccountAuth();

// 个人征信认证
void creditAuth();

// 工作认证
void occupationAuth();

// 第三方信用分认证
void thirdPartyAuth();

// 计算贷款认证分数
int calScore();

int getScore();
```

设计说明: AuthService是系统核心认证服务,整合了多种认证方式,为贷款审批提供基础数据。

3.2.4 AIApproveService (AI自动审核服务类)

属性:

```
private AuthService authService;
private ApproveMapper approveMapper;
private AIModelService aiModelService; // 机器学习模型API类
```

方法:

```
public void AICheck(); // AI自动审核
```

设计说明: AIApproveService是AI自动审核的核心服务,调用AuthService获取用户认证数据,通过机器学习模型进行自动化审批决策。

3.2.5 ManualApproveSendService (主管级审核员人工审核服务层)

属性：

```
private ManualApproveMapper manualApproveMapper; // 人工审核申请数据层
private UserMapper userMapper; // 提供筛选审核员的用户功能
```

方法：

```
public List<Approve> getApproves(); // 获得所有需审核申请
public Approve getApprove(); // 获得单个审核申请详情
public void manualCheck(); // 发送审核结果
```

设计说明：该服务负责人工审核流程的管理和分配，实现审核任务的自动化分配和处理。与 AIApproveService之间依靠消息队列进行通信。

3.3 关键交互流程

3.3.1 贷款申请流程

- 1. 用户提交贷款申请，包括个人信息、资产信息等
- 2. ApplicationService调用AuthService进行多源认证
- 3. AuthService整合资产认证、银行账户认证、征信认证等数据
- 4. RiskAssessService调用AIApproveService进行AI自动审核
- 5. 若AI审核通过，直接完成贷款审批；若不通过，调用ManualApproveSendService发送人工审核任务
- 6. 人工审核员通过ManualApproveService进行审核
- 7. 审核结果返回ApplicationService，生成贷款订单

3.3.2 贷后管理流程

- 1. 用户发起还款请求
- 2. OrderService调用PayService处理支付
- 3. 还款成功后，更新Loan状态和还款记录
- 4. 若用户申请延期，调用OrderService的postpone方法
- 5. 延期申请进入ManualApproveSendService流程，等待人工审核

3.4 数据设计

3.4.1 数据库表设计

用户表 (user)：

字段	类型	描述
user_id	INT	用户ID
username	VARCHAR(50)	用户名
password	VARCHAR(100)	密码

字段	类型	描述
phone	VARCHAR(20)	手机号
email	VARCHAR(50)	邮箱
create_time	DATETIME	创建时间

贷款产品表 (loan_product) :

字段	类型	描述
product_id	INT	产品ID
product_name	VARCHAR(100)	产品名称
interest_rate	DECIMAL(10,2)	利率
term_options	TEXT	期限选项
has_promotion	BOOLEAN	是否有优惠
promotion_details	VARCHAR(255)	优惠详情
min_amount	DECIMAL(10,2)	最低贷款金额
max_amount	DECIMAL(10,2)	最高贷款金额

贷款订单表 (loan_order) :

字段	类型	描述
order_id	INT	订单ID
user_id	INT	用户ID
product_id	INT	产品ID
amount	DECIMAL(10,2)	贷款金额
term	INT	期限 (月)
interest	DECIMAL(10,2)	利息
status	VARCHAR(20)	状态 (申请中、审批中、已放款、已结清)
create_time	DATETIME	创建时间
repay_time	DATETIME	还款时间

.....

3.4.2 数据交互格式

贷款产品DTO (LoanProductDTO) :

```
{
  "productName": "个人信用贷款",
  "interestRate": 0.05,
  "termOptions": [3, 6, 12],
  "hasPromotion": true,
  "promotionDetails": "新用户首贷优惠",
  "loanAmount": {
    "minAmount": 1000,
    "maxAmount": 50000
  }
}
```

用户认证结果:

```
{
  "userId": 1001,
  "assetScore": 85,
  "creditScore": 75,
  "bankAccountVerified": true,
  "occupationVerified": true,
  "thirdPartyScore": 90
}
```

.....

3.5 部署设计

系统采用微服务架构部署，各服务独立部署，通过Spring Cloud实现服务发现与负载均衡：

- 前端服务：html，css，js与后端一起静态资源部署
- 后端服务：
 - ├─ UserService（用户服务）
 - ├─ LoanProductService（贷款产品服务）
 - ├─ ApplicationService（申请服务）
 - ├─ OrderService（贷后服务）
 - ├─ PayService（支付服务）
 - ├─ AuthService（认证服务）
 - ├─ RiskAssessService（风险评估服务）
- 数据库：MySQL集群（主从复制）
- 缓存：Redis集群（用于会话管理和缓存热点数据）
- 消息队列：RabbitMQ（用于异步任务处理）

部署架构图：



4. 设计模式与复用机制

4.1 采用的设计模式

- 策略模式**：用于贷款审批策略，支持多种审批策略的动态切换
 - LoanApproveStrategy接口
 - AIApproveStrategy实现类
 - ManualApproveStrategy实现类
- 工厂模式**：用于创建不同类型的认证服务
 - AuthFactory类
 - AssetAuthStrategy实现类
 - BankAccountAuthStrategy实现类
- 观察者模式**：用于贷款状态变更通知
 - LoanStatusObserver接口
 - SmsService实现类（短信通知）
 - EmailService实现类（邮件通知）

4.2 复用机制

- 服务复用**：AuthService被多个服务（ApplicationService、RiskAssessService等）复用
- 工具类复用**：Utils包中的工具类（如JwtUtils、RedisUtils）,简化代码编写，被各服务模块复用
- 异常处理复用**：Exception包中的自定义异常类被各服务模块复用

5. 系统质量属性设计

5.1 性能设计

- 缓存机制**：对热门贷款产品信息、用户基本信息进行缓存，减少数据库访问
- 异步处理**：使用RabbitMQ处理短信通知、日志记录等非核心业务
- 数据库优化**：对贷款订单表进行分库分表，支持高并发查询

5.2 安全设计

- 认证安全**：使用JWT进行用户认证，确保请求安全
- 数据安全**：敏感数据（如身份证号、银行卡号）加密存储
- 权限控制**：基于角色的访问控制（RBAC），确保不同角色访问权限

5.3 可维护性设计

- 1. **模块化设计**：每个服务职责单一，便于独立开发和测试
- 2. **代码规范**：统一的代码规范和注释要求
- 3. **日志记录**：关键业务流程记录详细日志，便于问题排查

6 实施指南

6.1 人员分工

人员	职责
龚陈陈	负责文档撰写、设计、后端代码编写、bug解决
周飞凤	负责后端代码编写
秦雨欣	负责app端用户页面的代码编写
王婷	负责web端管理员页面的前端代码编写

6.2 开发顺序

- 需求分析
- 数据库结构设计
- 接口文档撰写
- 前后端编码
- 单元测试
- 集成测试

6.3 测试范围与测试方法

- 对所有网络请求进行测试
 - 后端进行postman测试，测试结果写入接口文档中
 - 前端进行请求测试，Web端控制台测试结果写入网络请求测试文档中
- 对核心业务流程进行测试
 - 用户注册登录流程测试
 - 用户查看贷款产品流程测试
 - 贷款申请流程测试
 - 贷款管理流程测试
 - 贷款审核流程测试
- 对中间件组件进行测试
 - Redis缓存测试
 - RabbitMQ消息队列测试
 - MySQL数据库测试
 - 日志记录测试

- 对系统性能进行测试
 - RabbitMQ消息队列的吞吐量、延迟、丢包率等指标进行测试
 - MySQL数据库的读写性能、连接池使用情况等指标进行测试
- 对系统安全进行测试
 - 对jwt的安全性进行测试

可维护规范声明

- 代码规范：遵循阿里巴巴Java开发手册、Google Java编程风格规范
- 注释规范：使用Javadoc注释，注释内容包括功能、参数、返回值、异常、注意事项等
- 日志规范：使用Log4j记录日志，日志内容包括时间、类名、方法名、日志级别、日志信息等

6. 结论与展望

本详细设计基于UML类图和概要设计文档，明确了系统的类结构、关键交互流程和数据模型。设计遵循了高内聚低耦合、可扩展性等原则，满足了系统性能和安全需求。

后续工作：

1. 实现各服务模块的具体代码
2. 编写单元测试和集成测试用例
3. 进行系统性能测试和安全测试
4. 优化关键路径的性能瓶颈

本设计为系统开发提供了清晰的指导，确保了系统开发的顺利进行。